

Mobile Money Transaction API Security & Design Report

1. Introduction to API Security

API security focuses on protecting application programming interfaces from unauthorized access, data breaches, and misuse. Since APIs often expose sensitive data and critical operations, proper authentication and authorization mechanisms are essential.

In this Mobile Money Transaction API, Basic Authentication is implemented to control access to transaction resources. Users must provide valid credentials (username and password) encoded in Base64 within the HTTP Authorization header. This ensures that only authenticated users can perform operations such as creating, viewing, updating, or deleting transactions.

Although Basic Authentication is simple to implement and useful for development and testing, it provides minimal security when used over plain HTTP. Without encryption, credentials can be intercepted, highlighting the importance of understanding both its strengths and weaknesses in API security design.

2. Documentation of API Endpoints

The Mobile Money Transaction API is a RESTful service that manages mobile money transactions using hardcoded in-memory data. It supports standard CRUD operations and runs locally without a database.

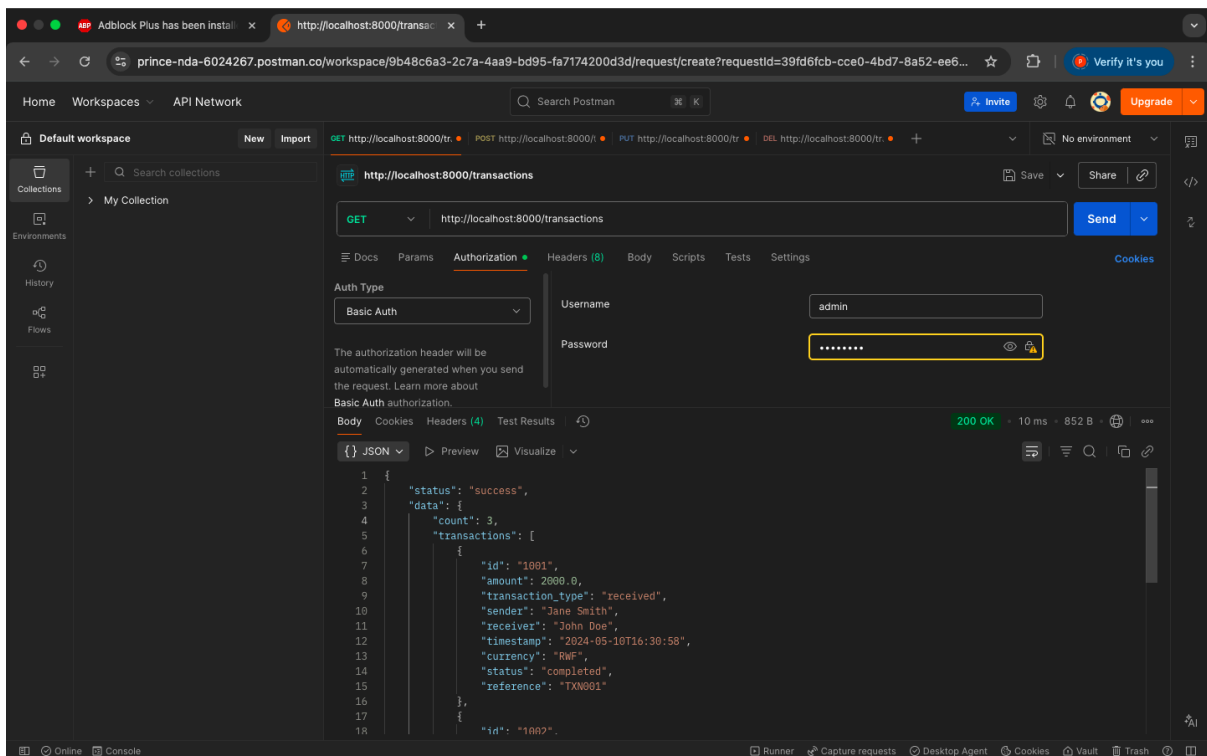
Authentication

- Method: **Basic Authentication**
- Test Users:
 - admin / admin123
 - user / user456
 - test / test789

All endpoints require valid authentication credentials.

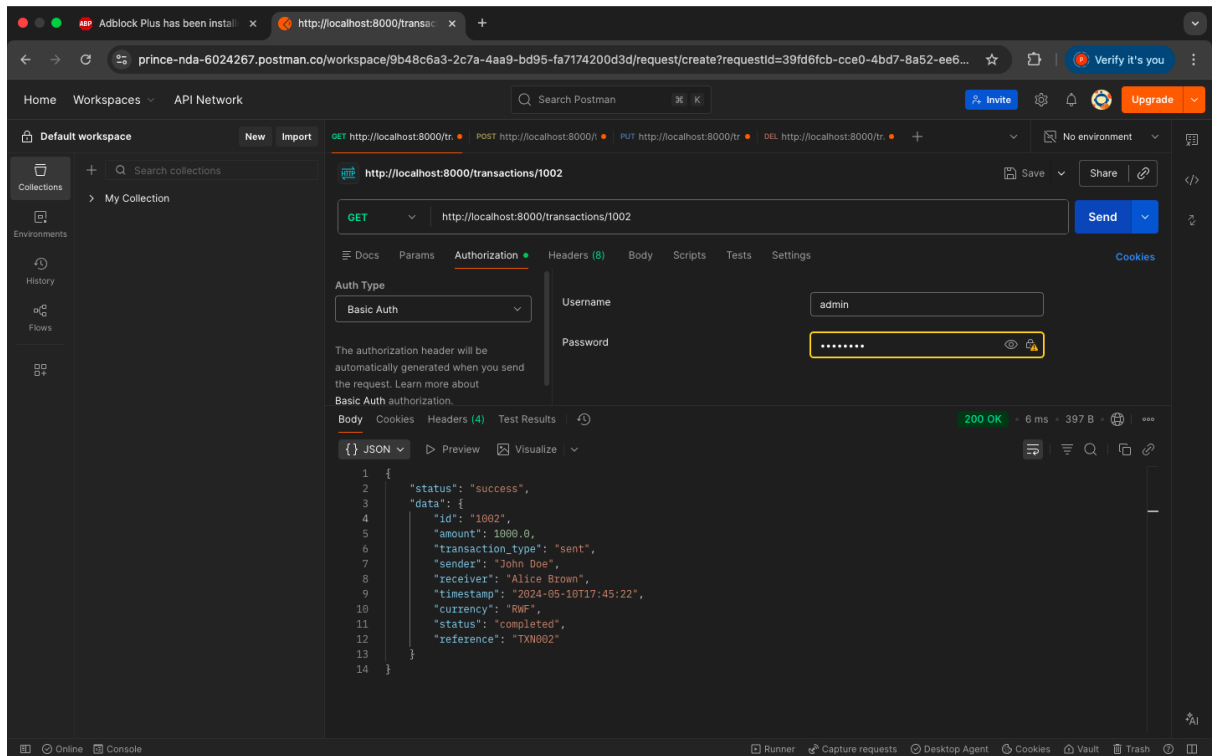
2.1 List All Transactions

- **Method:** GET
- **Endpoint:** /transactions
- **Description:** Retrieves all available transactions.
- **Response:** Returns the total count and a list of transaction objects.



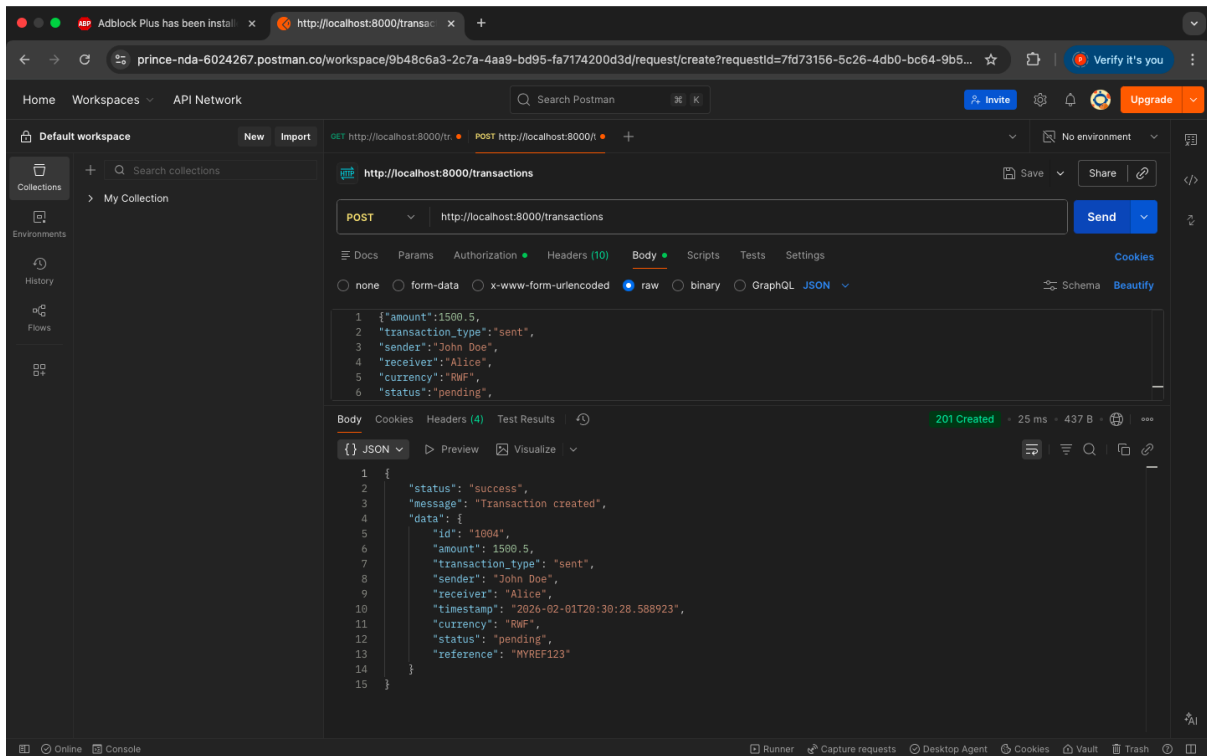
2.2 Get a Single Transaction

- **Method:** GET
- **Endpoint:** /transactions/{id}
- **Description:** Retrieves details of a specific transaction by ID.
- **Response:** Returns a single transaction object if found.



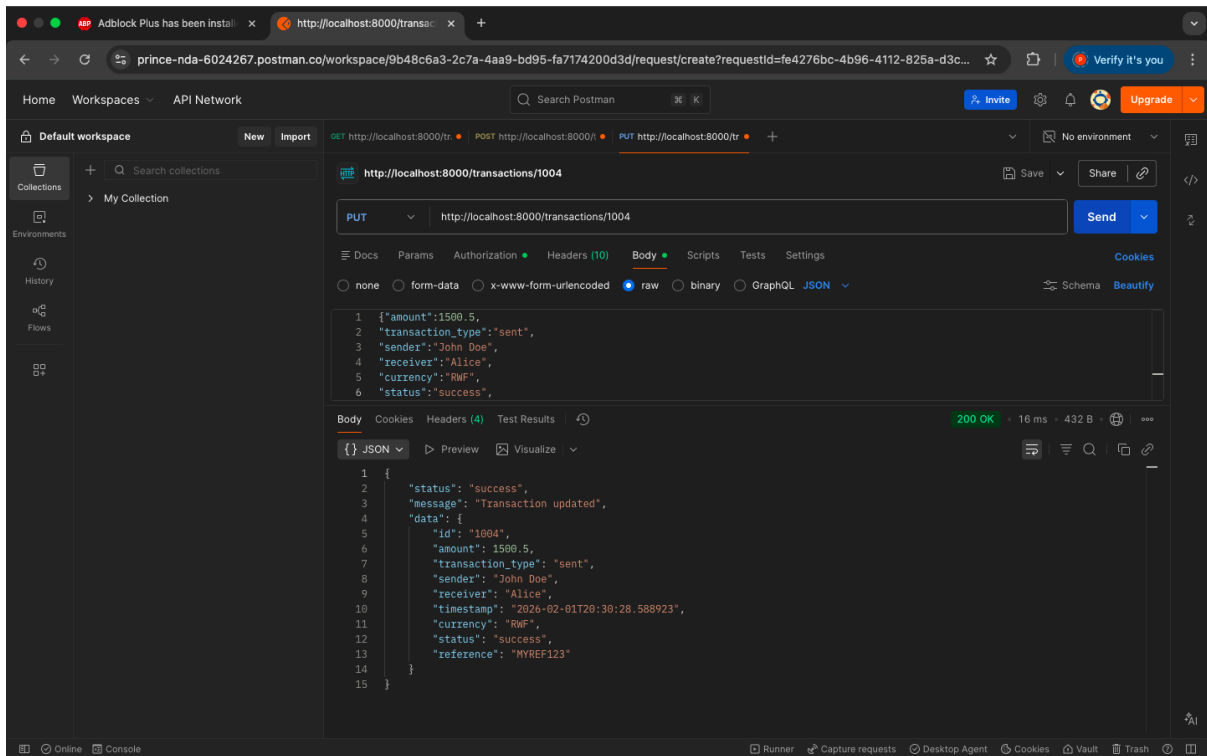
2.3 Create a New Transaction

- **Method:** POST
- **Endpoint:** `/transactions`
- **Description:** Creates a new mobile money transaction.
- **Required Fields:**
 - amount
 - transaction_type
 - sender
 - receiver



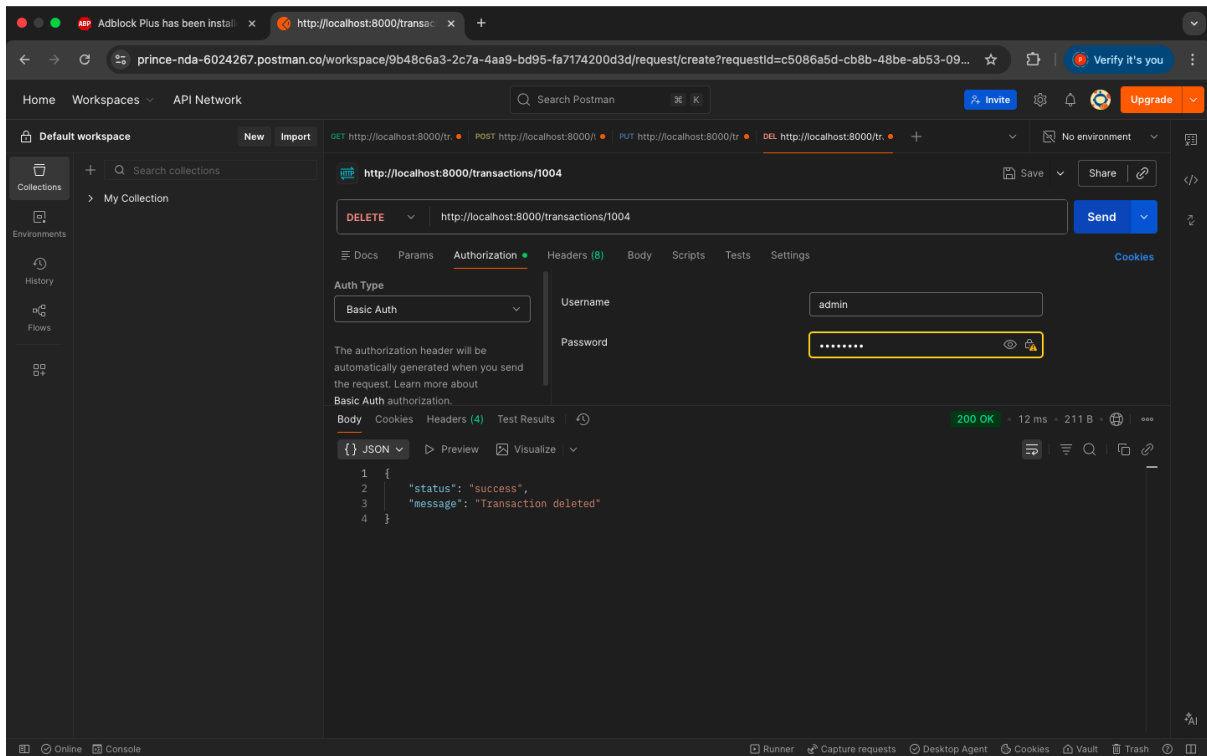
2.4 Update an Existing Transaction

- **Method:** PUT
- **Endpoint:** `/transactions/{id}`
- **Description:** Updates selected fields of an existing transaction.
- **Updatable Fields:**
amount, transaction_type, sender, receiver, currency, status, timestamp



2.5 Delete a Transaction

- **Method:** DELETE
- **Endpoint:** `/transactions/{id}`
- **Description:** Deletes a transaction permanently from memory.



Error Handling

- **401 Unauthorized:** Invalid authentication credentials
- **404 Not Found:** Transaction ID does not exist
- **400 Bad Request:** Invalid or malformed JSON data

3. Results of DSA Comparison

Data Structures and Algorithms (DSA) play a role in how transaction data is stored and accessed.

Current Implementation

- Transactions are stored in a Python dictionary
- Key: Transaction ID
- Value: Transaction object

Comparison Results

Data structure	Access Time	Suitability
Dictionary	$O(1)$	Best choice for fast lookup
List	$O(n)$	Inefficient for searches
Database Index	$O(\log n)$	Best for production systems

Result:

The dictionary-based approach provides constant-time access and is ideal for a lightweight, in-memory, development-only API. However, it lacks persistence and scalability compared to database-backed solutions.

4. Reflection on Basic Authentication Limitations

While Basic Authentication is easy to implement and understand, it has several critical limitations:

- Credentials are repeatedly sent with every request
- No encryption by default (unsafe without HTTPS)
- Vulnerable to interception and replay attacks
- No role-based access control
- Not suitable for production systems

In this API, Basic Authentication is acceptable for testing, learning, and local development, but it is not secure enough for real-world financial systems. More secure alternatives include:

- Token-based authentication (JWT)
- OAuth 2.0
- API keys combined with HTTPS

Conclusion

The Mobile Money Transaction API demonstrates a clear and functional REST design with authentication, structured endpoints, and efficient memory data handling. It successfully illustrates fundamental API security concepts and DSA usage. However, its reliance on

Basic Authentication and non-persistent storage limits it to educational and development environments rather than production use.